

GUIA RESUMEN — Spec Driven Development

Resumen integral para dominar SDD en ~30 minutos: patron paso a paso con plantillas por fase, teoria del libro, herramienta OpenSpec, metodologia aplicada a tu stack y auditoria de codigo generado por IA.

0

Como abordar una feature nueva — patron paso a paso

PASOS 0-8 · MODO ANDAMIAJE | COMPLETO

Este es el patron que distila los 6 casos practicos de **ejemplos-cambios/** y de **07-guia-paso-a-paso.md**. Sigue estos pasos en orden para **CUALQUIER feature nueva**. Dentro de cada plantilla, a la derecha de cada parte, hay un recordatorio de **que debe ir ahi**. Profundiza en **04-plantilla-cambio-openspec.md** (plantilla completa) y en **07-guia-paso-a-paso.md** (ejemplos llenos).

Hasta el Paso 5 no hay modo que elegir: **la IA redacta los artefactos y TU los apruebas**. La decision real llega en el Paso 6: el modo (**andamiaje | completo**) de **tasks.md** decide cuanto escribe la IA — siempre sobre el MISMO motor: **/opsx-apply-change**.

0 Clasifica tamano	1 Crea cambio carpeta	2-3 Proposal+Spec EARS	4 Design fase 2	5 Tasks modo+oleadas	6 Implementa apply-change	7-8 Verifica+Archiva specs vivas
--------------------------	-----------------------------	------------------------------	-----------------------	----------------------------	---------------------------------	--

Paso 0 · Clasifica el cambio antes de empezar

Elegir el tamano define cuanto ritual aplicas (proporcionalidad). **Regla practica:** si dudarias en mergearlo sin revision de otro humano, es al menos Intermedia.

```
$ openspec list          - cambios activos: ubicate antes de crear otro
```

Nivel	Criterio	design.md	Puertas	Ejemplo lleno en doc 07
Simple	1 feature, 0-1 tablas, 0 decisiones tecnicas nuevas	No	1 combinada	add-theme-color
Intermedia	1-2 features, 1-2 tablas, 1-2 decisiones tecnicas	Si (ligero)	1-2	add-user-profile
Compleja	3+ features, 3+ tablas, seguridad, cambios de contrato	Si (completo)	3	add-payments

Paso 1 · Crea el cambio

Dos modos de arranque. Ambos generan la misma carpeta y convergen en la Puerta 1.

Modo	Comando	Cuando
Artesano	/opsx-new-change	sabes QUE construir y quieres control fino del esqueleto
Copiloto	/opsx-propose add-nombre	la IA redacta la primera version de los 4 archivos

```
$ /opsx-new-change add-<feature>    - artesano: 4 ficheros vacios, los llenas TU
/opsx-propose add-<feature>         - copiloto: la IA redacta, tu apruebas
```

```
• • • openspec/changes/add-<feature>/

openspec/changes/add-<feature>/      - carpeta en kebab-case; prefijo add- para features nuevas
├─ proposal.md                      - Por que, impacto, alcance y actores (Paso 2)
├─ specs/
│   └─ <capacidad>/
│       └─ spec.md                  - requisitos EARS en formato delta (Paso 3)
├─ design.md                        - solo Intermedia/Compleja: arquitectura + ficheros (Paso 4)
└─ tasks.md                        - 5 oleadas + modo + trazabilidad Req„tarea„test (Paso 5)
```

Paso 2 · Llena proposal.md

Obligatorio en brownfield. Responde **POR QUE** existe el cambio, **QUE** toca de lo actual y **HASTA DONDE** llega.

```
$ openspec instructions proposal --change add-<feature> --json  - plantilla del artefacto
# rellena: WHY · Impact · Scope · Actores · Preguntas abiertas
```

```
• • • proposal.md

# Proposal: add-<feature>          - nombre del cambio en kebab-case (add- = feature nueva)
```

```
## Impacto (Impact Report)      ← QUE del sistema ACTUAL toca este cambio. OBLIGATORIO en brownfield
- Features afectadas: [que features de lib/ se afectan y como]      # ej: auth, cart
- Reutilizable: [contratos / patrones / widgets que YA existen]      # ej: Failure, Either
- Supabase: [tablas nuevas o existentes, politicas RLS, numeros de migracion]
- DI / rutas: [service_locator +N registros · app_router +N rutas]
- Riesgos: [los 2-3 principales: concurrencia, seguridad, breaking]

## Why (Problema)                ← necesidad real sin resolver HOY, en palabras del usuario (2-4 frases)
## What Changes (Solucion)       ← que construyes para resolver el Why, en lenguaje de dominio
## Capabilities                 ← capacidad afectada en kebab-case + 1 frase
## Scope (Alcance)              ← la frontera exacta del cambio
**Incluye:** [lista concreta de lo que SI se construye]
**No incluye:** [lo que EXPLICITAMENTE no se toca: pagos, envios, realtime...]
**Dependencias:** [que necesita del proyecto] · **Suposiciones:** [que das por hecho]
**Preguntas abiertas:** [pendientes; cierralas (tachado + respuesta) antes de la Puerta 1]

## Actores y permisos           ← tabla: Actor | Puede | No puede | Mapeo RLS
## Impact                      ← resumen: ~n ficheros · tablas/migracion · breaking (o "ninguno")
```

Paso 3 · Llena spec.md — requisitos EARS

La spec es el **contrato aprobado**. Cada **#### Scenario**: ES un criterio de aceptacion testeable. Cubre SIEMPRE: camino feliz + error + borde + aislamiento RLS.

```
$ openspec instructions spec --change add-<feature> --json      ← plantilla con deltas EARS
# cada #### Scenario: = 1 criterio de aceptacion testeable
```

• • • spec.md

```
# Spec: <capacidad>                ← nombre de la capacidad en kebab-case
## WHY                             ← por que existe esta capacidad (conecta con el Why del proposal)
## Purpose                         ← que garantiza esta capacidad, en 1-2 frases
## ADDED Requirements              ← formato delta: ADDED / MODIFIED / REMOVED
#### Requirement: [Nombre] (REQ-001) ← requisito ATOMICO con ID unico
[enunciado EARS: "el sistema DEBERA...", sin "deberia" ni ambigüedad]

#### Scenario: [Camino feliz]        ← cada escenario = 1 criterio de aceptacion
- **WHEN** [disparador] → **THEN** el sistema DEBERA [resultado observable] [+ AND si aplica]

#### Scenario: [Error]              ← mensaje EXACTO entre comillas; no altera estado
- **IF** [fallo esperable] → **THEN** el sistema DEBERA mostrar "[mensaje literal]"

#### Scenario: [Borde / aislamiento] ← limites numericos, vacios y permisos por usuario
- **GIVEN** [estado previo, ej. autenticado] → **WHEN** [accion] → **THEN** [solo lo propio: auth.uid() = user_id]
```

Puerta 1 · Validar requisitos

Ejecuta **openspec validate** y la **Clarity Gate**: ¿un dev senior entenderia la spec sin preguntar nada?

```
$ openspec validate add-<feature>      ← formato OK (schemas de los 4 archivos)
# y luego la Clarity Gate abajo
```

```
[ ] cada historia es precisa: actor + accion + objeto + proposito
[ ] requisitos con ID unico (REQ-XXX) y patron EARS correcto
[ ] bordes cubiertos: vacios, limites numericos, expirados, negativos
[ ] preguntas abiertas del alcance cerradas o documentadas
[ ] formato delta correcto (ADDED/MODIFIED/REMOVED) + Impact justifica el alcance
[ ] openspec validate pasa sin errores
```

Paso 4 · Llena design.md — disenio tecnico (solo Intermedia/Compleja)

En Simple se omite: los ficheros se derivan directamente de los requisitos. En Intermedia/Compleja se decide **como se implementa lo aprobado en la Puerta 1**.

```
$ openspec instructions design --change add-<feature> --json    ← plantilla del artefacto
# solo Intermedia/Compleja: en Simple se omite
```

• • • design.md

```
# Design: add-<feature>            ← nombre del cambio
## Context                        ← estado ACTUAL relevante + que cambia (1-2 oraciones)
## Goals / Non-Goals             ← objetivo claro (haz) / anti-objetivo explicito (NO haz)
## Decisions (D1..Dn)           ← solo decisiones NO obvias; una por bloque
```

```

### D1: [titulo]          ← Decision: [que se eligio]
                          ← Alternativas descartadas: [que se descarto y por que]
                          ← Por que: [razonamiento]

## Ficheros afectados     ← OBLIGATORIA: si falta, el agente INVENTA rutas
| Elemento | Capa | Archivo | Req | ← una fila por fichero, citando REQ
## Contratos Dart clave   ← Repository interface + sealed State ANTES de implementar
## Flujo de datos         ← ASCII: Page→Cubit→UseCase→Repo→DataSource→Supabase, con rama de error (Either.left)
## Backend Supabase       ← Tablas · RLS citando REQ · RPC si concurrencia · Migracion idempotente
## Boundaries             ← que NO debe hacer el agente aqui: ej. no tocar feature Y

```

Puerta 2 · Validar disen0

```
$ openspec validate add-<feature> -o disen0 estructuralmente valido (schemas)
```

```

[ ] la tabla de ficheros afectados cubre TODOS los REQ de la Puerta 1
[ ] los contratos compilan mentalmente (firmas coherentes entre capas)
[ ] el flujo de datos cubre caminos de exito Y de error
[ ] RLS especificada para toda tabla nueva/modificada
[ ] ninguna decision importante quedo implicita
[ ] openspec validate pasa sin errores

```

Paso 5 · Llena tasks.md — las 5 oleadas

Traduce el disen0 en pasos de implementacion. Cada tarea: **que se hace (≤3 ficheros) + REQ que implementa + criterio de exito + commit atomico**. Antes de las oleadas, declara el **modo de implementacion** (andamiaje | completo): quien escribe el codigo, siempre via `/opsx-apply-change`.

```

$ openspec instructions tasks --change add-<feature> --json -o oleadas + trazabilidad
# linea 2: declara el modo (andamiaje | completo) - decide cuanto escribe la IA

```

• • • tasks.md

```

# Tasks: add-<feature>
> Modo de implementacion: andamiaje ← quien escribe el codigo en este cambio
> andamiaje = scaffold + TODOS REQ, implementas TU · completo = IA escribe 100%, auditas
## 1. Dominio                ← logica pura: entidad, contrato y casos de uso
- [ ] 1.1 Entity <Name> + invariantes (REQ) · 1.2 Interface <Name>Repository (REQ)
- [ ] 1.3 UseCases (uno por operacion)
## 2. Capa de datos + Backend Supabase ← BD primero: soporta el flujo de datos desde el inicio
- [ ] 2.1 Migracion SQL tablas + RLS (REQ) (+ RPC / Edge Functions si aplica)
- [ ] 2.2 Model fromJson/toJson (snake_case→camelCase) · 2.3 DataSource (llamadas exactas)
- [ ] 2.4 RepositoryImpl (mapeo excepciones→Failure)
## 3. Estado y presentacion      ← estado visible y UI
- [ ] 3.1 State sealed · 3.2 Cubit (transiciones + mensajes EXACTOS de los escenarios)
- [ ] 3.3 Page + widgets (un render por estado)
## 4. Integracion                ← cableado de la app: DI y rutas
- [ ] 4.1 service_locator · 4.2 app_router
## 5. Tests                      ← 1 test por escenario critico citando REQ
- [ ] 5.1 Unit entidades/usecases · 5.2 Repository impl (mock) + integracion BD real · 5.3 Widget page clave
## Trazabilidad                  ← matriz | Req | Tarea(s) | Test | Cubre escenario |

```

Puerta 3 · Validar tareas

```
$ openspec validate add-<feature> -o tareas bien formadas y trazabilidad OK
```

```

[ ] cada requisito de la spec tiene al menos 1 tarea que lo implementa
[ ] cada tarea referencia su REQ (trazabilidad bidireccional)
[ ] las dependencias definen el orden correcto de oleadas
[ ] ninguna tarea excede ~3 ficheros · hay tests para escenarios criticos
[ ] tasks.md declara su modo (andamiaje | completo)
[ ] openspec validate pasa sin errores

```

Paso 6 · Implementa — un solo motor: /opsx-apply-change

En los Pasos 0-5 (y hasta la Puerta 3) solo redactaste y aprobaste artefactos. Aqui **/opsx-apply-change** ejecuta las tareas de tasks.md oleada por oleada; el **modo** declarado en tasks.md (Paso 5) decide cuanto escribe la IA y cuanto implementas tu.

Criterio	Modo andamiaje	Modo completo
Que hace la	scaffold por tarea (throw UnimplementedError() + TODO citando REQ), casilla en ☐	implementa cada tarea al 100% contra los escenarios y marca ☐

IA	<code>] , pausa</code>	<code>[x]</code>
Tu rol	implementas los bodies; escribes la logica critica del dominio	auditas cada diff contra la spec
Ideal para	logica critica (dinero, permisos, estados) · aprender stack · spec ambigua	CRUD, formularios, brownfield quirurgico, specs bien especificadas
Ciclo por oleada	IA aplica scaffold → implementas → tests corren → ajustas	IA escribe → auditas vs spec → cumple? sigue : fix citando REQ+escenario

```
$ /opsx-apply-change      ← unico motor; respeta el Modo declarado en tasks.md
```

El modo se declaro en tasks.md (Paso 5): andamiaje = el agente genera scaffold por tarea (UnimplementedError + TODOs REQ), deja la casilla en `- [ ]` y pausa; TU implementas los metodos criticos — completo = la IA escribe 100% y marca `- [x]` ; tu auditas. Para que `/opsx-apply-change` respete el modo de forma fiable, configura UNA vez `operations.apply.guidance` en `openspec/config.yaml` del proyecto (ver 04-plantilla).

Paso 7 · Verifica y Paso 8 · Archiva

Antes de cerrar: tests verdes, matriz llena, mensajes = literales exactos de la spec, contratos intactos.

```
$ /opsx-verify-change      ← valida la implementacion contra la spec
flutter test              ← tests verdes (matriz llena, mensajes literales)
openspec validate --archived ← todo el formato sigue OK
```

```
$ /opsx-archive-change     ← consolida los deltas en openspec/specs/ (specs vivas)
openspec list --specs      ← confirma: la capacidad ya esta descrita hoy
```

Regla del libro: puedes delegar la ESCRITURA, nunca la APROBACION de la spec (Puerta 1) ni la verificacion final (Puerta 3).

Si encontraste un bug al verificar: ¿bug de CODIGO? → fix citando REQ+escenario. ¿Bug de la SPEC? → corrígela primero con `/opsx-update-change` y re-aplica; no fuerces el codigo a cumplir un requisito mal escrito.

Flujo en una mirada — fase → comando → puerta

Fase	Comando	Puerta / Resultado
0 · Clasifica	<code>openspec list</code>	decides Simple / Intermedia / Compleja
1 · Crea	<code>/opsx-new-change <nombre> · /opsx-propose add-nombre</code>	carpeta <code>openspec/changes/<nombre>/</code>
2 · proposal	<code>openspec instructions proposal --change <nombre> --json</code>	Puerta 1 + Clarity Gate
3 · spec	<code>openspec instructions spec --change <nombre> --json</code>	validada en Puerta 1: <code>openspec validate</code>
4 · design	<code>openspec instructions design --change <nombre> --json</code>	Puerta 2: <code>openspec validate</code>
5 · tasks	<code>openspec instructions tasks --change <nombre> --json</code>	Puerta 3: <code>openspec validate</code>
6 · Implementa	<code>/opsx-apply-change (modo andamiaje completo)</code>	tests verdes por oleada
7 · Verifica	<code>/opsx-verify-change · flutter test</code>	Puerta 3 FINAL: matriz llena
8 · Archiva	<code>/opsx-archive-change</code>	<code>openspec list --specs</code> (specs vivas)

1 Teoria libro	2 Herramienta OpenSpec	3 Metodologia stack	4 Practica ejemplos	5 Auditoria codigo IA
----------------------	------------------------------	---------------------------	---------------------------	-----------------------------

Regla de oro: "La especificacion es el contrato. El codigo cambia, la spec describe lo que el sistema HACE ahora mismo."

Paso	Que aprendes	Archivo / recurso
1. Teoria	Fases, puertas, EARS, deltas (libro SDDEquiposAgiles)	01-teoria-sdd.md + pdf/
2. Herramienta	OpenSpec CLI: init, slash commands, ciclo del cambio	03-openspec-guia-practica.md
3. Metodologia	SDD aplicado a Flutter + Clean Arch + Supabase	02-sdd-flutter-supabase.md
4. Practica	6 cambios completos de Simple a Compleja	ejemplos-cambios/
5. Auditoria	Verificar codigo escrito por IA contra la spec	06-auditoria-codigo-ia.md

2 Que es Spec Driven Development (3 min)

LIBRO · SDDEQUIPOSAGILES_V.1.PDF

SDD = escribir primero la especificacion ejecutable del comportamiento, aprobarla como contrato, y derivar de ella el codigo, los tests y la documentacion. La IA amplifica el problema: genera mucho codigo con poca comprension; la spec devuelve el control.

Los tres pilares

Pilar	Que es	Artefacto
Teoria	El libro: fases, puertas de calidad, lenguaje comun	pdf/SDDEquiposAgiles_v.1.pdf
Herramienta	OpenSpec CLI: carpeta de cambio por feature	openspec/changes/<cambio>/
Metodologia	Adaptacion al stack: que artefacto toca cada capa	02-sdd-flutter-supabase.md

El cambio de rol del desarrollador

Antes (vibe coding)	Con SDD
Prompt → codigo → rezar	Spec aprobada → codigo → verificar contra spec
"Funciona en mi maquina"	Cumple los escenarios EARS o no se mergea
Documentacion obsoleta al dia siguiente	Specs vivas: describen el sistema ACTUAL

Tres beneficios: Alineamiento antes de codear · Trazabilidad requisito→codigo→test · Onboarding y auditoria triviales.

3 Las 4 fases y las 3 puertas (4 min)

NUCLEO DEL METODO

Fase	Entregable	Puerta de calidad
1. Especificar	proposal.md + spec delta (requisitos EARS)	Puerta 1: ¿la spec es precisa y completa? ¿un dev senior la entiende sin preguntar?
2. Diseñar	design.md (ficheros, contratos Dart, backend, decisiones)	Puerta 2: ¿cada flecha del flujo tiene artefacto? ¿decisiones justificadas?
3. Tareas	tasks.md (oleadas, trazabilidad Req↔tarea↔test)	(incluida en Puerta 2)
4. Implementar	Codigo + tests + commits atomicos por tarea	Puerta 3: tests verdes, matriz llena, Clarity Gate, archive

Clarity Gate: ¿un agente distinto, con SOLO la spec, produciria codigo equivalente? Si dudas, hay supuestos ocultos → corregir la spec AHORA (todavia es barato).

Proporcionalidad: no todo merece el ritual completo

Tamano	Esfuerzo spec	Ejemplo
Simple	proposal + spec, puerta combinada	cambiar un label validado, ajuste de query
Intermedia	las 4 fases completas	nueva pantalla CRUD, add-buyers
Compleja	4 fases + Impact Report + boundaries explicitos	facturacion, delivery realtime, pagos

Cada requisito = **ID + oracion EARS + escenarios**. Sin "deberia", sin ambigüedades: el sistema DEBERA, punto.

Patron	Palabra clave	Cuando usarlo	Ejemplo (add-cart)
Event-driven	WHEN ... THE SYSTEM SHALL	evento dispara comportamiento	WHEN agrega producto con stock > 0 THEN agregar cantidad 1 y congelar precio
State-driven	WHILE <estado>	comportamiento segun estado persistente	MIENTRAS carrito tenga items, mostrar subtotal/impuesto/total
Unwanted	IF ... THEN SHALL	casos de error y excepciones	IF cupon expiro THEN "El cupon {codigo} ha expirado"
Optional	WHERE <feature>	solo si el feature esta activo	WHERE cupones activados, aplicar descuento
Complex	GIVEN ... WHEN ... THEN	precondiciones multiples	GIVEN cliente autenticado WHEN consulta su carrito THEN solo filas auth.uid()=user_id

Clasificacion de reglas → patron EARS

Tipo de regla	Significado	Patron tipico
RN (regla negocio)	invariantes del dominio (tope 50% descuento)	Unwanted / Event
RT (regla transicion)	estados y transiciones validas	State-driven
RS (regla sistema)	limites tecnicos (rate limit, tamano)	Unwanted

• • • openspec/changes/add-cart/

openspec/changes/add-cart/

- └─ proposal.mdWHY + WHAT Changes + Impact
- └─ specs/
 - └─ shopping-cart/
 - └─ spec.mdDelta: ADDED / MODIFIED / REMOVED Requirements
- └─ design.mdFicheros afectados · Contratos Dart · Backend · Decisiones
- └─ tasks.mdOleadas + trazabilidad Req+task+test

Operador delta	Semantica	Efecto en archive
## ADDED Requirements	capacidad nueva	se anaden al main spec
## MODIFIED Requirements	requisito existente cambia	reemplaza la version anterior
## REMOVED Requirements	requisito ya no aplica	se elimina del main spec

Tras el archive: **openspec/specs/** contiene las especificaciones vivas = descripcion actualizada del sistema. Nunca documentacion muerta.

Slash command	Que hace	Cuando
/opsx-explore	explorar una idea antes de especificarla	fase de discovery
/opsx-propose <nombre>	genera proposal + spec delta + design + tasks	inicio de todo cambio
/opsx-new-change	carpeta de cambio manual desde plantilla	control fino
/opsx-continue-change	retoma el cambio donde quedo	sesiones largas
/opsx-update-change	edita artifacts del cambio en curso	tras feedback de Puerta 1
/opsx-apply-change	implementa tareas oleada por oleada	Paso 6 · unico motor (modo completo andamiaje)
/opsx-verify-change	valida implementacion contra la spec	Puerta 3
/opsx-archive-change	consolida deltas en specs vivas	al aprobar todo
/opsx-sync-specs · /opsx-onboard · /opsx-bulk-archive-change	mantenimiento de specs	ocasional

CLI directa	Uso
openspec init [dir]	inicializar en un proyecto (una vez)
openspec list · openspec list --specs	ver cambios activos / capacidades
openspec validate <cambio> --strict	validar formato del cambio
openspec instructions <proposal spec design tasks apply> --change <n> --json	plantilla / guia del artefacto
openspec view · openspec change <nombre>	dashboard / detalle

Setup en 2 comandos: `npm i -g @fission-ai/openspec && openspec init` — luego reinicia el editor para cargar los slash commands (/opsx-*).

Dos estilos de arranque: /opsx-propose = modo copiloto (la IA redacta, tu apruebas). /opsx-new-change = modo artesano (esqueleto vacio; lo llenas TU y la IA refina con /opsx-update-change + validate --strict). Ambos convergen en la Puerta 1.

La spec dice	Vive en	Capa Clean Arch
entidades y calculos puros	domain/entities/*.dart	Domain
reglas de negocio (RN)	usecases / entity	Domain
contratos de datos	domain/repositories/*.dart (Either<Failure,T>)	Domain
fuentes de datos, RPCs	data/datasources/ + data/models/ (snake_case)	Data
estados UI y transiciones	presentation/cubit/ (sealed states)	Presentation
mensajes exactos de escenarios	CartError(mensaje literal), nunca en UI	toda la cadena
tablas, RLS, migraciones	supabase/migrations/NNNN_*.sql idempotente	Backend

Reglas Supabase dentro de la spec

- ✓RLS habilitada en toda tabla nueva; policias por actor × operacion declaradas en design.md \$Backend
- ✓concurrency critica (stock, secuencias) → RPC security definir atomico, no SELECT+UPDATE en cliente
- ✓aislamiento entre usuarios = escenario EARS obligatorio (GIVEN ... auth.uid() = user_id)

Oleadas estandar de tasks.md (oleadas = capas): **O1 Dominio (entity + interface + usecases) → O2 Capa de datos + Backend Supabase (migracion SQL + RLS + RPC → model → datasource → repository impl) → O3 Estado + UI (state + cubit + page) → O4 Integracion (DI + rutas) → O5 Tests. La migracion abre la O2 para que el flujo de datos sea probable desde el inicio. Commit atomico por tarea.**

A Andamiaje scaffold por tarea + pausa	C Completo IA escribe 100%
--	--

Ambos modos corren sobre el MISMO motor: /opsx-apply-change — la diferencia es el modo declarado en `tasks.md`

Criterio	Modo andamiaje	Modo completo
Logica critica: dinero, permisos, estados	escribiste cada linea (no pierdes la practica)	⚠ solo con escenarios EARS exhaustivos
CRUD, formularios, brownfield quirurgico	posible overkill	terreno ideal
Aprendiendo stack o dominio	implementar ES aprender	x no aprendes nada
Specs ambiguas (falla Clarity Gate)	implementar resuelve dudas	x el error se multiplica

Regla del libro: puedes delegar la escritura, **NUNCA** la aprobacion de la spec (Puerta 1) ni la verificacion final (Puerta 3). El modo se declara en `tasks.md` (Paso 5) y el motor es SIEMPRE `/opsx-apply-change`. La skill `clean-arch-feature` queda como herramienta manual opcional (fuera del flujo) si prefieres generar scaffolding a mano.

- ✓**Contratos intactos:** repository interface = design.md verbatim; Either<Failure,T> en todo; cero try/catch tragador; nada fuera de la tabla de ficheros
- ✓**Sealed states:** switch exhaustivo sin caso _; mensajes = literales EXACTOS de los escenarios ("Producto {nombre} sin stock disponible" ≠ "Stock insuficiente")
- ✓**Capa correcta:** RN en domain, no en cubit/UI; usecases en domain/, repository impl en data/; una sola fuente de verdad cliente/servidor; decisiones D1..Dn respetadas
- ✓**Supabase ⚠:** RLS en todas las tablas nuevas; polices = design.md; RPC security definer revisado linea por linea; cero service_role en cliente; migracion idempotente e igual al design
- ✓**Tests contra spec:** 1 test por Scenario citando REQ; asserts usan literales; cero tests siempre-verdes
- ✓**Trazabilidad:** matriz llena; cero UnimplementedError() huérfanos; commit atomico por tarea
- ✓**Red flags:** diff fuera de alcance; dependencias nuevas sin justificar; over-engineering; boundaries violados ("No desactivar RLS jamas")

Ciclo por oleada: agente aplica → tu auditas diff vs spec → cumple ✓ sigue / no cumple → fix citando REQ+escenario. Si el bug era de la SPEC, corrígela primero (/opsx-update-change) y re-aplica.

Equivalencia	Ahora (SDD / OpenSpec)
Descomposicion de feature	Carpeta de cambio (proposal + specs + design + tasks)
Historias de usuario + criterios Gherkin	Requirements con ID + escenarios EARS
Definition of Done / sprint review	Puertas de calidad (Puerta 1, 2, 3)
Product Backlog refinado	Main specs (openspec/specs/) — sistema actual
Nuevo sprint para cambiar algo	Deltas ADDED / MODIFIED / REMOVED
Mapeo de diseno a capas	design.md \$Ficheros afectados (Elemento Capa Archivo Req)

Cambio	Capacidad	Complejidad	Lo que demuestra
add-cart	shopping-cart	Intermedia ★ walkthrough	flujo completo paso a paso; RPC atomico stock; precio congelado; RLS por usuario
add-buyers	buyers-management	Simple → Intermedia ★★	puerta combinada; aprobar/liberar tickets; minimo ritual suficiente
approve-reservas	appointments	Compleja ★★★	doble confirmacion; reagendar con politica 24h; pg_cron; realtime
add-elearning-progress	enrollment-progress	Intermedia ★★	gating de lecciones via RLS; vista de progreso; trigger auto-completada
add-facturacion	invoicing	Compleja ★★★	maquina de estados invoice; secuencias fiscales; RPC transaccional; notas credito
add-delivery-seguimiento	delivery-tracking	Compleja ★★★	GPS realtime broadcast; tarifa PostGIS; timeout de entrega con cron

Como practicar: lee add-cart completo junto a 02-sdd-flutter-supabase.md → copia la estructura a openspec/changes/ en tu proyecto → adapta. Luego intenta escribir tu propio cambio ANTES de pedirle la spec a la IA.